

High Tech Marketing Text Analysis

Brand: Motorola Razr 2025

Group 12?

Introduction

Explain shit here philippe 8=====D

Sample Selection

We decided to go with multiple sources of text in an attempt to capture multiple perspectives on the Razr and avoid pigeon holing into one corner of the internet which may have skewed results.

Data Sources

We grabbed our shit from Reddit threads about the Motorola Razr 2025 (pain in my ass) and Youtube comments from popular tech youtubers who reviewed the Razr 2025

Sampling representation & Bias

Explain if the sample is representative of all customers, and how platform choice may skew results e.g Reddit chuds are not normal people.

Data Collection

```
fetch_reddit_thread <- function(thread_url, limit = 500, attempts = 5) {  
  api_url <- paste0(thread_url, ".json?limit=", limit)  
  response <- RETRY(  
    "GET",  
    api_url,  
    user_agent(paste0("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ",  
                      "AppleWebKit/537.36 (KHTML, like Gecko) ",  
                      "Chrome/125.0.0.0 Safari/537.36")),  
    times      = attempts,  
    pause_base = 2,  
    pause_cap  = 20,  
    terminate_on = c(400, 401, 403, 404)  
  )  
}
```

```

if (http_error(response)) {
  stop("HTTP ", status_code(response), " when fetching ", api_url)
}

payload <- fromJSON(
  content(response, as = "text", encoding = "UTF-8"),
  simplifyVector = FALSE
)
post <- payload[[1]]$data$children[[1]]$data

flatten_comments <- function(children) {
  if (!is.list(children)) return(data.frame())
  rows <- list()
  for (child in children) {
    if (!is.null(child$kind) && child$kind == "t1") {
      comment <- child$data
      body_text <- comment$body %||% NA_character_
      if (!is.null(body_text)) body_text <- gsub("[\r\n]+", " ", body_text)
      rows[[length(rows) + 1]] <- data.frame(
        type = "comment",
        id = comment$id %||% NA_character_,
        parent_id = comment$parent_id %||% NA_character_,
        author = comment$author %||% NA_character_,
        body = body_text,
        score = comment$score %||% NA_real_, # keep score for segmentation
        subreddit = comment$subreddit %||% NA_character_, # keep subreddit for segmentation
        stringsAsFactors = FALSE
      )
      replies <- if (is.list(comment$replies) &&
        !is.null(comment$replies$data$children))
        comment$replies$data$children else list()
      if (length(replies) > 0)
        rows <- c(rows, flatten_comments(replies))
    }
  }
  if (length(rows) == 0) return(data.frame())
  rows <- Filter(function(x) is.data.frame(x) && ncol(x) > 0, rows)
  if (length(rows) == 0) return(data.frame())
  tryCatch(
    dplyr::bind_rows(rows),
    error = function(e) {
      cols <- c("type", "id", "parent_id", "author", "body", "score", "subreddit")
      repaired <- lapply(rows, function(r) {
        if (!is.data.frame(r)) return(NULL)
        for (m in setdiff(cols, names(r))) r[[m]] <- NA
        r[cols]
      })
      dplyr::bind_rows(Filter(Negate(is.null), repaired))
    }
  )
}

comments <- flatten_comments(payload[[2]]$data$children %||% list())

```

```

post_body <- post$selftext %||% post$title %||% NA_character_
if (!is.null(post_body)) post_body <- gsub("[\r\n]+", " ", post_body)

post_df <- data.frame(
  type      = "post",
  id        = post$id      %||% NA_character_,
  parent_id = NA_character_,
  author    = post$author %||% NA_character_,
  body      = post_body,
  score     = post$score  %||% NA_real_,
  subreddit = post$subreddit %||% NA_character_,
  stringsAsFactors = FALSE
)

if (!is.data.frame(comments)) comments <- data.frame()
tryCatch(
  dplyr::bind_rows(post_df, comments),
  error = function(e) post_df
)
}

suppressWarnings({
  reddit_cache <- "data/reddit_content.csv"
  cached <- file.exists(reddit_cache)

  if (cached) {
    reddit_all <- tryCatch(
      read.csv(reddit_cache, stringsAsFactors = FALSE),
      error = function(e) { message("Cache read failed: ", conditionMessage(e)); data.frame() }
    )
  }
  else {
    search_config <- list(
      list(sub = "Android",      kw = "motorola razr"),
      list(sub = "Android",      kw = "razr 2025"),
      list(sub = "Android",      kw = "razr flip"),
      list(sub = "Motorola",      kw = "razr"),
      list(sub = "Motorola",      kw = "razr ultra"),
      list(sub = "Motorola",      kw = "razr plus"),
      list(sub = "phones",       kw = "razr"),
      list(sub = "phones",       kw = "motorola"),
      list(sub = "smartphones",  kw = "motorola razr"),
      list(sub = "razr",         kw = "razr"),
      list(sub = "razr",         kw = "motorola"),
      list(sub = "technology",   kw = "motorola razr"),
      list(sub = "technology",   kw = "razr 2025")
    )

    reddit_posts <- dplyr::bind_rows(Filter(
      function(x) is.data.frame(x) && nrow(x) > 0, # add this Filter wrapper
      lapply(search_config, function(cfg) {
        result <- tryCatch({
          Sys.sleep(2)

```

```

    find_thread_urls(keywords = cfg$kw, subreddit = cfg$sub, sort_by = "top")
  }, error = function(e) {
    message("Failed r/", cfg$sub, " / '", cfg$kw, "': ", conditionMessage(e))
    data.frame(url = character()) # always return a data frame, never NULL
  })
  cat("r/", cfg$sub, " / '", cfg$kw, "' ->",
      if (is.data.frame(result)) nrow(result) else "ERROR", "threads\n")
  # Ensure we always return a data frame regardless of what find_thread_urls gives back
  if (!is.data.frame(result)) return(data.frame(url = character()))
  result
})
)) |> dplyr::distinct(url, .keep_all = TRUE)

cat("Total unique threads:", nrow(reddit_posts), "\n")

reddit_max_threads <- 250

if (nrow(reddit_posts) > 0) {
  to_fetch <- head(reddit_posts$url, reddit_max_threads)
  raw_list <- lapply(seq_along(to_fetch), function(i) {
    Sys.sleep(1)
    message("Thread ", i, "/", length(to_fetch))
    tryCatch(
      fetch_reddit_thread(to_fetch[i]),
      error = function(e) {
        message("Failed: ", conditionMessage(e))
        data.frame()
      }
    )
  })
  reddit_all <- dplyr::bind_rows(
    Filter(function(x) is.data.frame(x) && nrow(x) > 0, raw_list)
  )
} else {
  reddit_all <- data.frame()
}

write.csv(reddit_all, reddit_cache, row.names = FALSE)
})
})

cat("Total rows:", nrow(reddit_all), "\n")

```

```
## Total rows: 992
```

```
cat("Posts vs comments:\n")
```

```
## Posts vs comments:
```

```
print(table(reddit_all$type))
```

```
##
```

```
## comment      post
##      765      227
```

```
cat("By subreddit:\n")
```

```
## By subreddit:
```

```
print(table(reddit_all$subreddit))
```

```
##
##      Android      motorola      phones      razr Smartphones
##           6          312          29          608          37
```

```
fetch_youtube_comments <- function(video_id, max_comments = 200, api_key = Sys.getenv("YOUTUBE_KEY")) {
  if (identical(api_key, "")) {
    stop("YOUTUBE_KEY is not set. Load it from .env or your environment before knitting.")
  }

  all_comments <- list()
  page_token <- NULL

  repeat {
    query <- list(
      part      = "snippet",
      videoId   = video_id,
      maxResults = 100,
      textFormat = "plainText",
      key       = api_key
    )

    # Add pagination token if we have one
    if (!is.null(page_token)) {
      query$pageToken <- page_token
    }

    resp <- httr::GET(
      "https://www.googleapis.com/youtube/v3/commentThreads",
      query = query
    )

    if (httr::http_error(resp)) {
      error_body <- tryCatch(
        httr::content(resp, as = "text", encoding = "UTF-8"),
        error = function(e) ""
      )
      message("YouTube API error: ", httr::status_code(resp), if (nzchar(error_body)) paste0(" - ", error_body))
      break
    }

    parsed <- jsonlite::fromJSON(
      httr::content(resp, as = "text", encoding = "UTF-8"),
      flatten = TRUE
    )
  }
}
```

```

)

# Extract top-level comments
items <- parsed$items

if (is.null(items) || nrow(items) == 0) break

batch <- data.frame(
  comment_id = items$id,
  author      = items$snippet.topLevelComment.snippet.authorDisplayName,
  body        = items$snippet.topLevelComment.snippet.textDisplay,
  stringsAsFactors = FALSE
)

all_comments[[length(all_comments) + 1]] <- batch
collected <- sum(sapply(all_comments, nrow))
message("Collected: ", collected, " comments")

# Check for next page
page_token <- parsed$nextPageToken

if (is.null(page_token) || collected >= max_comments) break

Sys.sleep(0.5)
}

if (length(all_comments) == 0) return(data.frame())
dplyr::bind_rows(all_comments) |> dplyr::mutate(source = "youtube");
}

razr_videos <- list(
  mkbhd      = "8om1eJr021U",    # MKBHD Razr 2025 review
  dave2d     = "JTT67FSc8j8",    # ShortCircuit review
  mrmobile   = "YGTkjchlVJk",    # MrMobile review
  phonearena = "b3Ijuf7DHcQ"    # PhoneArena review
)

# to save on api calls, we check if we have a local csv of comments already, if not we fetch and save f
if (file.exists("data/youtube_comments.csv")) {
  youtube_all <- read.csv("data/youtube_comments.csv", stringsAsFactors = FALSE)
  cat("Loaded YouTube comments from CSV:", nrow(youtube_all), "\n")
} else {
  youtube_all <- purrr::map2_dfr(
    razr_videos,
    names(razr_videos),
    ~ fetch_youtube_comments(.x, max_comments = 200) %>%
      dplyr::mutate(channel = .y)
  )
  # Save to CSV for future runs
  write.csv(youtube_all, "data/youtube_comments.csv", row.names = FALSE);
}

```

```
## Loaded YouTube comments from CSV: 704
```

```
cat("YouTube comments collected:", nrow(youtube_all), "\n")
```

```
## YouTube comments collected: 704
```

```
print(table(youtube_all$channel))
```

```
##
##      dave2d      mkbhd      mrmobile phonearena
##      200        200        200          104
```

Exploration

Lets have a gander at the data we collected and see if theres any thing fun before we clean it up.

```
# reddit
str(reddit_all)
```

```
## 'data.frame':   992 obs. of  7 variables:
## $ type      : chr  "post" "comment" "comment" "comment" ...
## $ id       : chr  "1sz3as2" "oiyrxch" "oj3o8r3" "oj7x93l" ...
## $ parent_id: chr  NA "t3_1sz3as2" "t3_1sz3as2" "t3_1sz3as2" ...
## $ author   : chr  "FragmentedChicken" "BeerorCoffee" "box-art" "z28camaroman" ...
## $ body     : chr  "" "I like the Razr but that pricing is just wild. " "Insane pricing for a phone ...
## $ score    : int  20 10 16 5 4 1 279 78 16 3 ...
## $ subreddit: chr  "Android" "Android" "Android" "Android" ...
```

```
# youtube
str(youtube_all)
```

```
## 'data.frame':   704 obs. of  9 variables:
## $ comment_id: chr  "Ugw-9HIGvJrOZ1iZZyx4AaABAg" "UgwMtrWVbmFWifzG4r54AaABAg" "UgxASY7bEBKG3vLuRsh4...
## $ author    : chr  "@chrishulsey2813" "@brettgirl" "@WalterSobchack-s8n" "@DTow-p1x7x" ...
## $ body      : chr  "Man I know this is old but can you review the newer ultras with 1tb storage and ...
## $ like_count: int  0 0 0 0 0 0 1 0 0 0 ...
## $ reply_count: int  0 0 0 0 0 0 0 0 0 0 ...
## $ date      : chr  "2026-05-03T12:24:10Z" "2026-04-28T15:45:50Z" "2026-04-02T05:27:44Z" "2026-03-30...
## $ source    : chr  "youtube" "youtube" "youtube" "youtube" ...
## $ segment   : chr  "neutral" "neutral" "neutral" "neutral" ...
## $ channel   : chr  "mkbhd" "mkbhd" "mkbhd" "mkbhd" ...
```

Pre-Processing

we have 2 separate data frames for reddit, and youtube, we need to merge them into one big one for the text analysis. We can add a column to indicate the source of each row and then bind them together.

```
##      [,1]
## de      1
## en 1556
```

```
## es      8
## fr      1
## pt      6
## ro      1
## ru      1
```

Side note maybe we dont have to do all of them based on the quality of the data we get, but we should at least explain how we would do them and why they are important.

Filtering & Cleaning

```
#remove non english comments
combined_data <- combined_data |> filter(language == "en")

#remove shit
combined_data$body <- gsub("[^\x01-\x7F]", "", combined_data$body)

#remove empty bodies
combined_data <- combined_data |> filter(body != "")
```

Modeling

Corpus creation

```
corpus <- quantda::corpus(combined_data, docid_field = "id", text_field = "body")
summary(corpus, 10)
```

Corpus consisting of 1556 documents, showing 10 documents:

```
##
## Text Types Tokens Sentences author score source language
## 2 11 11 1 BeerorCoffee 10 Android en
## 3 14 15 1 box-art 16 Android en
## 4 51 71 3 z28camaroman 5 Android en
## 5 18 22 2 Slowpc 4 Android en
## 6 11 13 1 the-Vibe 1 Android en
## 7 12 13 2 MKTECH3 279 motorola en
## 8 15 19 2 Powerful_District_82 78 motorola en
## 9 17 19 3 LoETR9 16 motorola en
## 10 6 6 1 dscdrivercpm-fr 3 motorola en
## 11 63 84 2 astralsparkle 3 motorola en
```

```
as.character(corpus[ sample(length(corpus), 1) ])
```

```
##
## "A few things I can answer from my Ultra 2025. Security updates are usual 2-4 months behind e.g. I'l
```


Tokenization

Explain how we tokenized the data and why.

Stop Words Removal

Explain how we removed stop words and why.

Stemming & Lemmatization

Frequency Filter

Synonym consolidation

Summary Statistics

Topic Modeling (LDA)

Build Document

Optimal K

Fit Model

Stability

Results

Topic Word Distributions

Sometother shit

Reflection